基于CUDA的BMP图像卷积池化



2025

问题回顾

- 任务：对BMP图像进行高斯卷积处理，实现图像模糊处理
- 输入图像：timg.bmp（RGB三通道，8位深度）
- 卷积核：5×5的高斯核，标准差1.5
- 边缘处理：零填充
- 最大池化：2×2窗口
- 原始尺寸：4096×2304
- 池化后尺寸：2048×1152

对比任务

架构对比

- **CPU架构（Pthread/OpenMP）：** 基于共享内存的MIMD架构（多指令多数据），CPU核数少，但是每个核心更强大，拥有大缓存
- **GPU架构（CUDA）：** 基于SIMT（单指令多线程），使用大量的线程组（warp）并行执行相同的指令，但每个线程处理不同的数据

任务划分

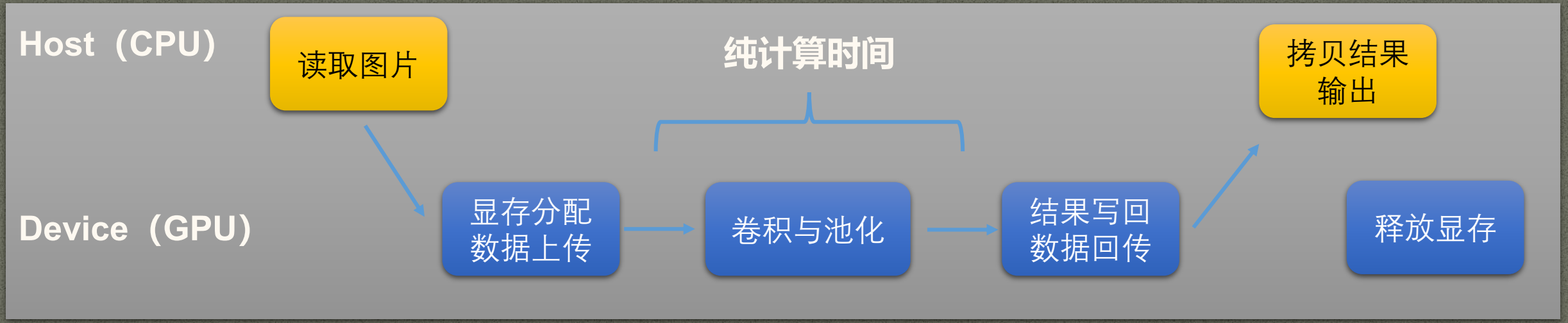
- **CPU：** 静态划分或指令自动拆分循环
- **GPU：** 二维网格映射

异构计算模型

Host与Device

Host（CPU）：读取 BMP 文件、计算 Grid/Block 尺寸、分配显存 、发起数据传输

Device（GPU）：执行核函数，卷积与池化计算，利用 SIMT特性同时处理



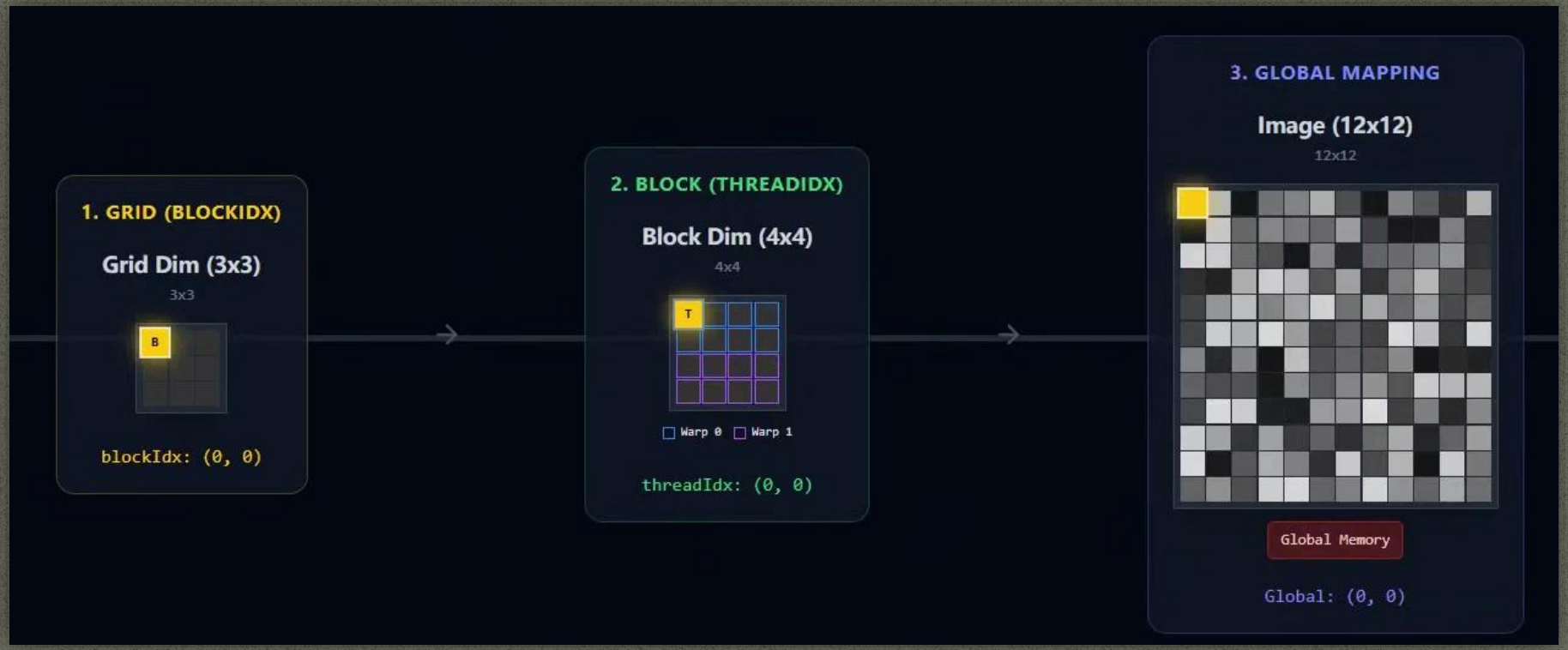
SIMT线程层次结构

逻辑层次：Grid-Block-Thread 三级架构

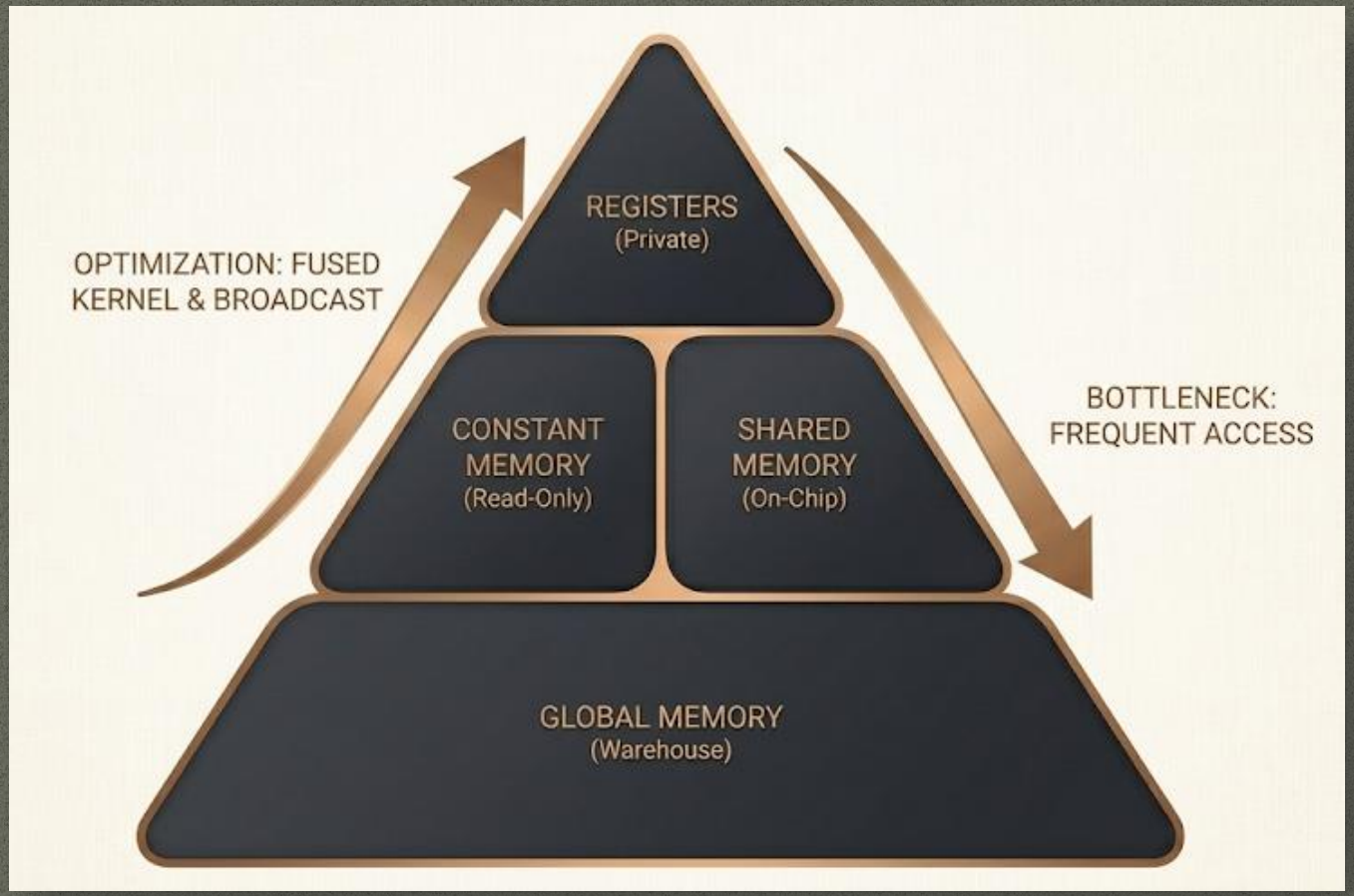
Grid（网格）： 对应整张图像的处理范围，覆盖所有像素

Block（线程块）： 线程块内的线程可以共享共享内存

Thread（线程）： 一个线程对应一个像素



内存结构



全局内存：容量最大，延迟最高，存放图像数据

常量内存：只读缓存，存放卷积核，广播机制

共享内存：Block的内存

寄存器：用于存储线程私有的数据，每个线程都有自己的寄存器

关键代码分析

计算图像尺寸

```
// 准备 Device 内存
Pixel *d_src, *d_conv, *d_pool;
size_t img_size = width * height * sizeof(Pixel);
size_t out_size = out_width * out_height * sizeof(Pixel);

CHECK(cudaMalloc(&d_src, img_size));
CHECK(cudaMalloc(&d_conv, img_size));
CHECK(cudaMalloc(&d_pool, out_size));

// 拷贝数据 Host -> Device
CHECK(cudaMemcpy(d_src, img.data.data(), img_size, cudaMemcpyHostToDevice));
CHECK(cudaMemcpyToSymbol(d_KERNEL, h_KERNEL, 25 * sizeof(float)));
```

卷积核定义为常量内存

cudaMalloc分配显存，手动管理内存

上传数据到gpu

```
__constant__ float d_KERNEL[25];

const float h_KERNEL[5][5] = {
```

关键代码分析Pthread

```
__global__ void convolution_kernel(Pixel* input, Pixel* output, int width, int height) {
    int x = blockIdx.x * blockDim.x + threadIdx.x;
    int y = blockIdx.y * blockDim.y + threadIdx.y;

    if (x >= width || y >= height) return;

    float sum_r = 0.0f, sum_g = 0.0f, sum_b = 0.0f;

    // 卷积计算
    for (int ky = -K_RADIUS; ky <= K_RADIUS; ++ky) {
        for (int kx = -K_RADIUS; kx <= K_RADIUS; ++kx) {
            int ny = y + ky;
            int nx = x + kx;

            if (ny >= 0 && ny < height && nx >= 0 && nx < width) {
                Pixel p = input[ny * width + nx];
                float w = d_KERNEL[(ky + K_RADIUS) * 5 + (kx + K_RADIUS)];
                sum_r += p.r * w;
```

计算全局坐标

对每个像素进行卷积，

读取全局内存，写入全局内存

越界保护，避免非法访存

零填充处理

结果性能分析

并行部分时间：纯 GPU 计算时间（不含传输），反映 GPU 算力。

整体运行时间：GPU 计算 + 显存分配 + 数据传输，反映加速卡处理该任务的真实延迟。

端到端时间：读取硬盘图片 + 上述所有步骤 + 写入硬盘图片，反映总等待时间

测试组号	并行部分时间（ms）	整体运行时间 (ms)	端到端运行时间 (ms)
1	**0.2232**	89.217	210.806
2	**0.2293**	44.551	169.884
3	**0.2371**	59.946	181.97

结果性能分析

- 计算占比极低，GPU 仅花费了不到 0.4% 的时间在执行卷积和池化运算，而超过 99.6% 的时间消耗在了 Host 与 Device 之间的数据传输以及显存管理上

- 基础版算法将原图传入 GPU，计算出中间结果（卷积图），再读取中间结果计算池化，虽然计算瞬间完成，但数据的搬运受限于传输总线带宽和 Global Memory 的访问延迟

- 用分步策略（Conv Kernel + Pool Kernel），每次启动 Kernel 都有微秒级的 CPU-GPU 通信开销

优化尝试

1. **尝试可分离卷积，降低计算复杂度**

$$C = A \otimes B = \begin{pmatrix} a_1b_1 & a_1b_2 & \cdots & a_1b_n \\ a_2b_1 & a_2b_2 & \cdots & a_2b_n \\ \vdots & \vdots & \ddots & \vdots \\ a_mb_1 & a_mb_2 & \cdots & a_mb_n \end{pmatrix}$$

2. **分析：**在 GPU 架构下，计算量的减少未能转化为加速，原因是该算法必须分两步执行（行卷积 -> 列卷积），迫使系统增加了一个额外的中间显存缓冲区**（Row Kernel + Col Kernel + Pool Kernel）**

$$A = \begin{pmatrix} a_1 & a_2 & \cdots & a_m \end{pmatrix} \quad B = \begin{pmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{pmatrix}$$

3. 结论：对于这种小卷积核，GPU 的计算能力是过剩的，而显存带宽是稀缺的，可分离卷积虽然省了计算，却增加了显存读写。因此，优化方向由减少计转向减少访存。

```
=== Performance Report (2350489) ===
1. 并行部分时间 (GPU Kernel) : 0.371712 ms
2. 整体运行时间 (H2D+Comp+D2H): 87.6904 ms
3. 端到端时间 (含硬盘 IO)    : 209.005 ms
========================================
```

优化策略

算子融合：边卷积边池化

用计算换带宽（减少访存）

基础版：卷积与池化完全分离，需要把卷积结果完整写入内存，再把结果从内存完整取出，多了访存开销，N个像素，就有2N次读写

由一个线程负责一个输入像素 → 一个线程负责生成一个最终输出像素

数据卷积后不存入显存，直接在寄存器中比较和丢弃

优化策略

单线程内的 "卷积 + 池化" 流水线 | 寄存器级优化

Global Memory (输入图像)



红色框: 2x2 池化窗口
蓝色区: 5x5 卷积感受野

单个 GPU 线程 (Thread)

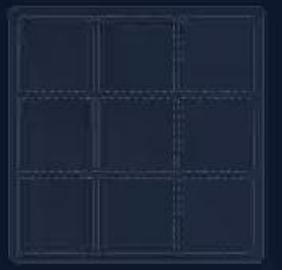
步骤1: 连续计算4次卷积

寄存器堆
(REGISTERS)

R0 –
R1 –
R2 –
R3 –

步骤2: 寄存器内比对 (Max Pooling)

Global Memory (最终输出)



性能优势分析:

- 显存写入次数: ~~4次~~ → 1次
- 显存读取次数: ~~中间结果读取~~ → 0次
- 数据交换: 全部在 片上寄存器 完成

优化代码分析

遍历2×2的池化窗口

```
// 局部循环：遍历 2x2 的池化窗口
#pragma unroll
for (int dy = 0; dy < 2; ++dy) {
    #pragma unroll
    for (int dx = 0; dx < 2; ++dx) {

        // 当前卷积中心点
        int cx = src_start_x + dx;
        int cy = src_start_y + dy;

        float sum_r = 0.0f, sum_g = 0.0f, sum_b = 0.0f;

        // 执行标准 5x5 卷积
        if (cx >= 0 && cx < in_width && cy >= 0 && cy < in_height) {
            #pragma unroll
            for (int ky = -K_RADIUS; ky <= K_RADIUS; ++ky) {
                #pragma unroll
                for (int kx = -K_RADIUS; kx <= K_RADIUS; ++kx) {
                    int nx = cx + kx;
                    int ny = cy + ky;

                    // 边界检查
                    if (nx >= 0 && nx < in_width && ny >= 0 && ny < in_height) {

                        Pixel p = input[ny * in_width + nx];

                        float w = d_KERNEL_2D[(ky + K_RADIUS) * 5 + (kx + K_RADIUS)];
                        sum_r += p.r * w;
```

循环展开

寄存器缓存

写入寄存器

```
if (sum_r > max_r) max_r = sum_r;
if (sum_g > max_g) max_g = sum_g;
if (sum_b > max_b) max_b = sum_b;
```

卷积完成后马上比较丢弃

结果性能分析

GPU 核心计算时间从平均 0.23ms 稳定下降至 0.18ms 左右，获得了约 1.28倍 的内核加速比

测试组号	并行部分时间（ms）	整体运行时间 (ms)	端到端运行时间 (ms)
1	0.1748	85.562	207.819
2	0.1894	81.787	212.653
3	0.1772	88.043	208.276

结果性能分析

- 虽然单线程的计算负载增加了，每个线程计算 4 次卷积，但由于数据直接在寄存器中流转，削减了对全局内存的中间写入和读取
- 对于 5×5 小卷积核任务，显存带宽是比计算算力更紧缺的资源，减少访存比减少计算更能带来性能提升
- GPU编程中，访存比计算更昂贵

感谢聆听

2025